

Buildbarn

A distributed build cluster

Ed Schouten <ed@nuxi.nl>

<https://github.com/buildbarn>



History of Bazel

Timeline (1/4)

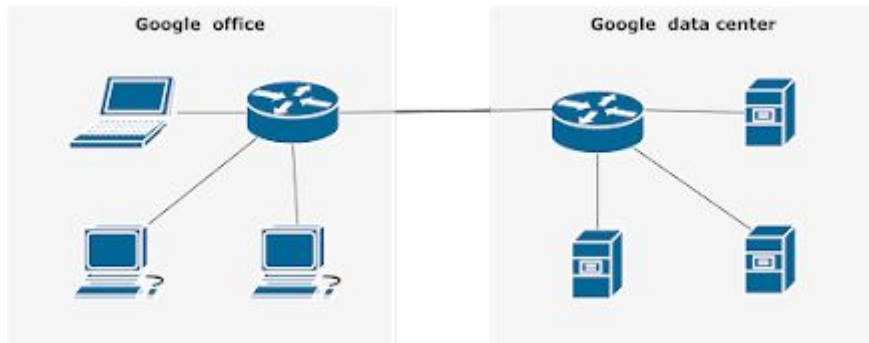
- ~2000: Google has a monorepo with shell script/Makefile build scripts.
 - It turns out that becomes unmaintainable relatively quickly.
- ~2005: Makefiles are replaced with build tool written in Python.
 - Every 'package' (directory) contains a BUILD file that is `eval()`ed by Python.
 - Directives are Python function calls that are implemented by the build tool.

```
cc_library(  
    name = "stringformatter",  
    srcs = ["stringformatter.c"],  
    hdrs = ["stringformatter.h"],  
)
```

```
cc_binary(  
    name = "hello",  
    srcs = ["hello.c"],  
    deps = [":stringformatter"],  
)
```

Timeline (2/4)

- ~2010: Blaze: rewrite of Python build tool in Java.
 - Contains a primitive Python interpreter to parse existing BUILD files.
 - `java_*`(), `cc_*`(), `py_*`(), etc. rules are all implemented inside Blaze in Java.
 - Sandboxing: actions only 'see' files that are part of their deps = [...].
 - Remote caching/execution: 'blaze -j 1000' from behind your desk.



Timeline (3/4)



- 2015: Bazel: tidied up Open Source version of Blaze.
 - Not extensible: mainly just `java_*`(), `cc_*`() and `py_*`() rules.
 - No remote execution: existing version was too Google specific.
- 2015-2023: Many new features appear.
 - Support for platforms other than Linux/x86, and a good notion of cross compilation.
 - Starlark: use a Python-like language to design your own build rules.
 - Support for fetching and source code and build rules remotely (HTTP, Git, etc.).

```
rust_library = rule(  
    _rust_library_impl,  
    attrs = {  
        "srcs": attr.label_list(),  
        "deps": attr.label_list(),  
    })
```

```
def _rust_library_impl(ctx):  
    ctx.actions.run("rustc", ...)  
    return [DefaultInfo(...)]
```

Timeline (4/4)

- Bazel gains support for remote caching and execution.
 - 2017: Initial 'RE' protocol was designed by Google.
 - 2018: Community efforts later on led to the release of 'REv2'.
- Servers that implement REv2 start to appear:
 - 2017: Uber releases Bazel Buildfarm, written in Java.
 - 2018: Bloomberg & CodeThink release BuildGrid, written in Python.
 - 2018: I started working on Buildbarn, written in Go.
- Clients other than Bazel that use REv2 start to appear:
 - Drop-in replacements for /usr/bin/cc: recc, Goma.
 - Existing build systems add REv2 support: Pants, Please, BuildStream, Buck.



‘distcc/ccache/... did this two decades ago’

- ... except that it only works for C/C++ compilation.
 - REv2 supports remote execution of arbitrary UNIX commands.
- ... except that it requires that workers have toolchains/SDKs preloaded.
 - REv2 allows clients to upload full SDKs to workers.
 - Workers can be vanilla OS installations.
 - Result: easier to achieve reproducibility of work.
- ... except that it only speeds up builds.
 - REv2 can also run unit/integration tests remotely and cache results.

REv2 is not a fad!

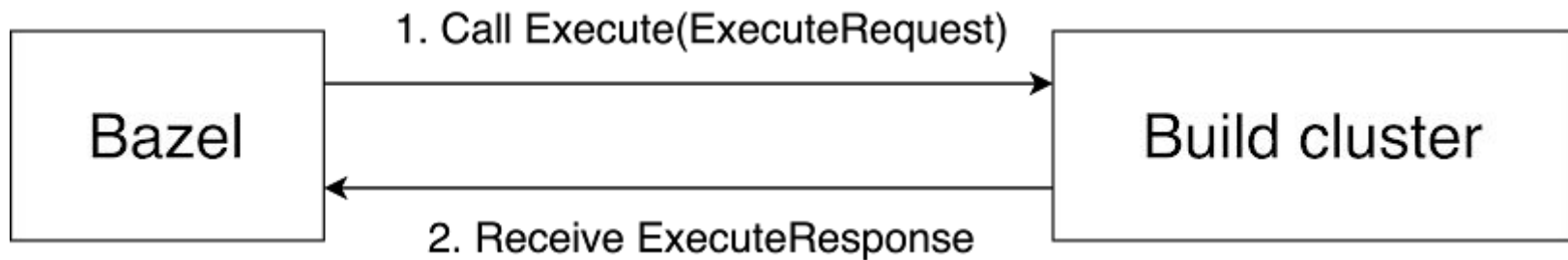
It is the de facto standard for distributed software builds.





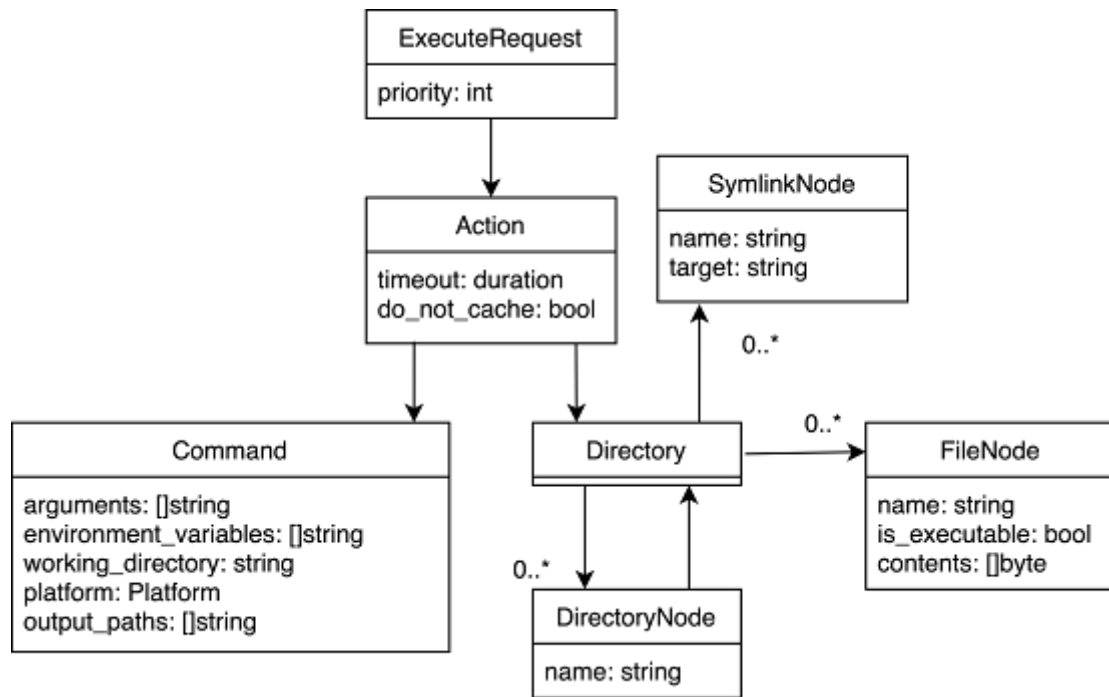
REv2

Remote Execution... simplified

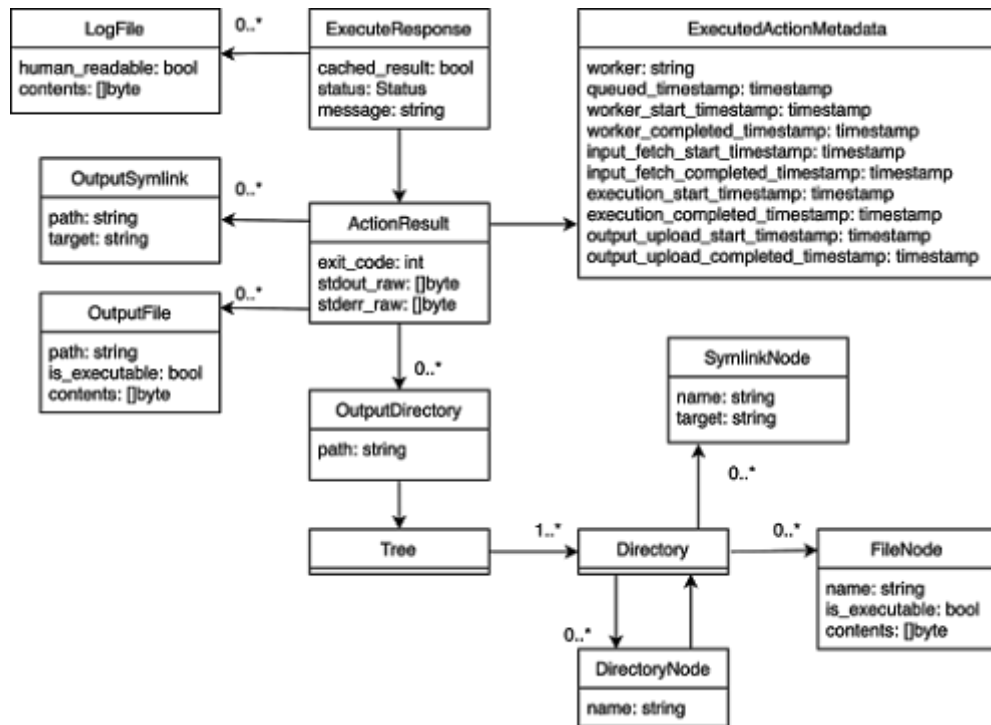


Building a project consists of hundreds/thousands of these calls.

ExecuteRequest



ExecuteResponse



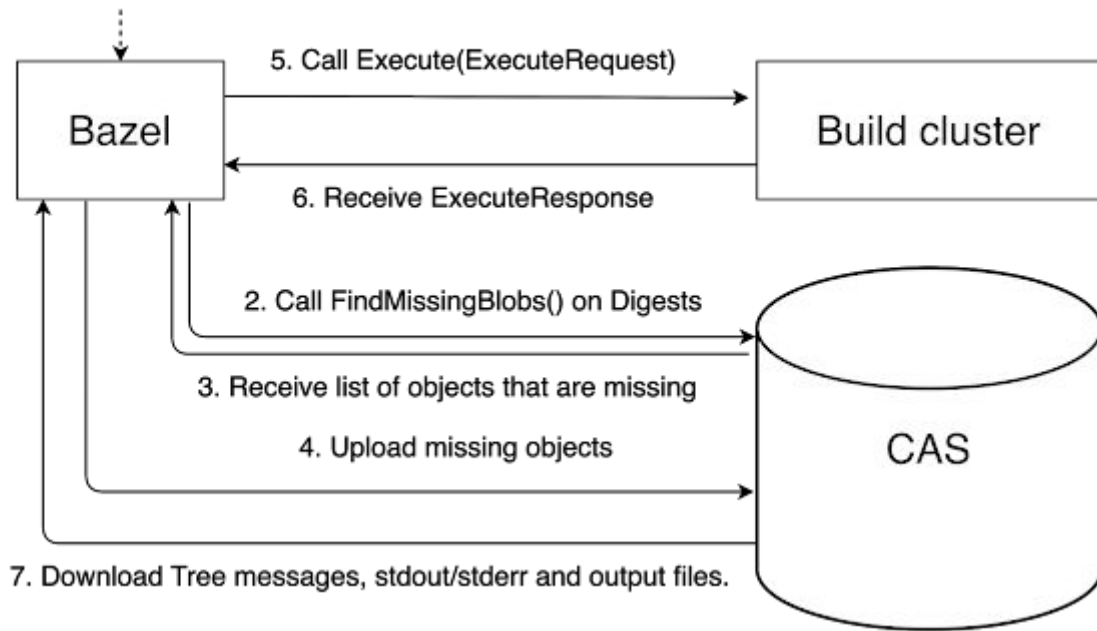
Content Addressable Storage (CAS)

- **Problem:** ExecuteRequest and ExecuteResponse get big and repetitive.
 - Input roots with SDKs can be hundreds of MBs in size.
 - Build-edit-build cycles create nearly identical ExecuteRequests.
- **Solution:** place repetitive parts in shared storage.
 - ExecuteRequest: Action, Command, Directory messages and file contents stored externally.
 - ExecuteResponse: Tree messages and (log)file contents stored externally.
 - Use content addressing: objects are identified by a Digest (i.e., SHA-256 + size).
 - Automatic deduplication of identical data.
 - Tamper proof Merkle tree: contents can be validated when loaded.
 - Immutability of data makes caching trivial.



Remote Execution with the CAS

1. Compute ExecuteRequest, Action, Command and Directories messages.



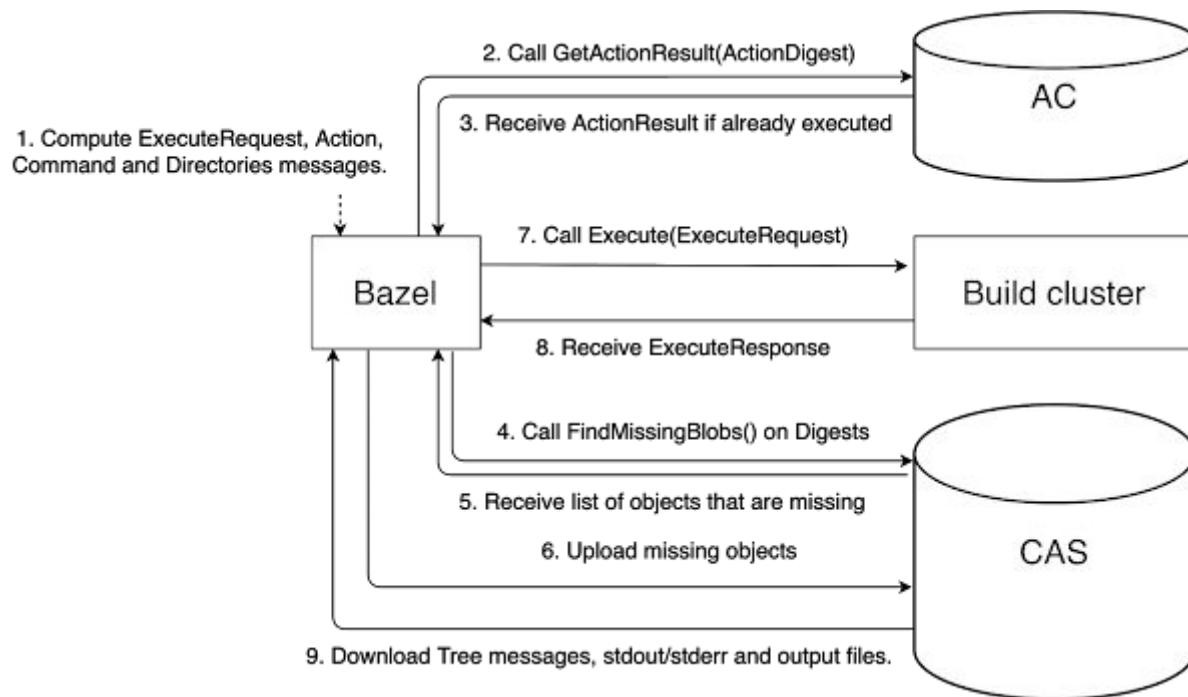
Note: only communication involving Bazel is shown.

Action Cache (AC)

- **Problem:** protocol is still expensive for builds that are already cached.
 - At least two round-trips: FindMissingBlobs() and Execute().
 - FindMissingBlobs() size grows linear w.r.t. input root file count.
- **Solution:** let the client first query the Action Cache directly.
 - GetActionResult(Digest of the Action) → ActionResult.
 - AC size is minuscule compared to the CAS: about 1/1000th the size.
 - AC is the only part of REv2 storage that can become poisoned.



Remote Execution with the CAS & AC



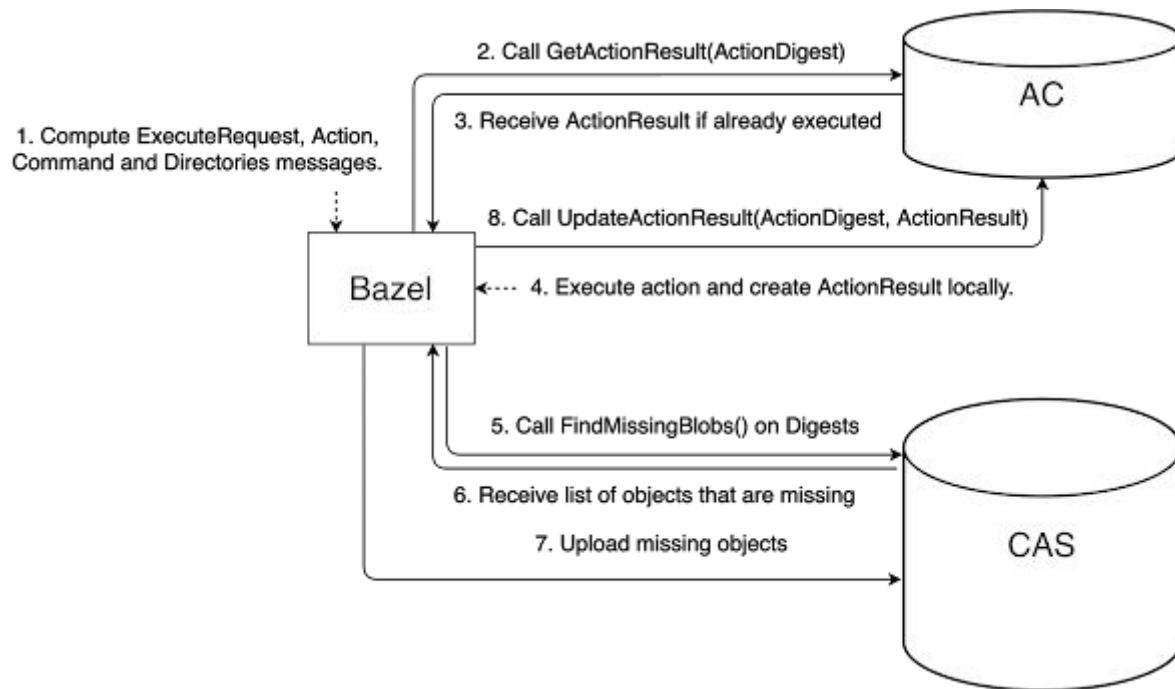
Note: steps 4 to 8 are skipped in case step 3 returns success.

Remote Caching: still execute actions locally

- **Problem:** Full Remote Execution may not always be feasible.
 - If a small number of build actions don't work with Remote Execution yet.
 - If setting up Remote Execution infrastructure requires too much maintenance.
- **Solution:** Let Bazel execute locally and write to the AC directly.
 - Can be set up in BUILD files on a per-target basis if needed.
 - Does potentially allow users to poison the AC.
 - Hint: Only allow CI systems to write into the AC. Users can still read the AC.



Remote Caching: still execute actions locally



Note: steps 4 to 8 are skipped in case step 3 returns success.

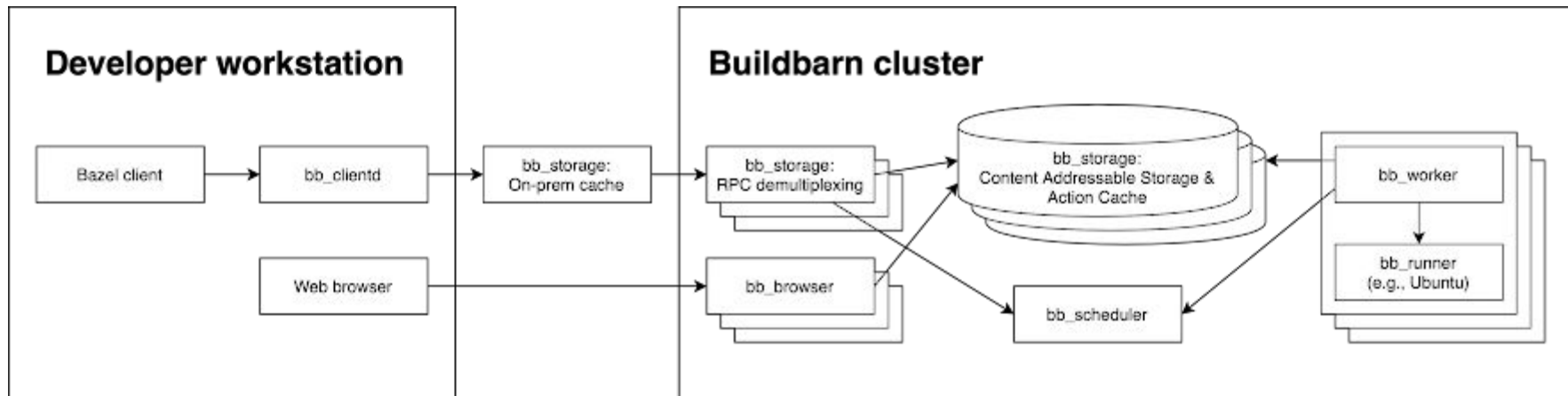
REv2 Summary

- RPC to run commands remotely: **Execute()**.
 - The client sends ExecuteRequests.
 - References an Action, a Command, Directories, and individual files.
 - The cluster sends back ExecuteResponses.
 - References an ActionResult, Trees, and individual files.
- Data store #1: **Content Addressable Storage (CAS)**.
 - Stores Actions, Commands, Directories, Trees and individual files.
 - Big. Cannot be poisoned. Safe to provide write access to all workers and clients.
- Data store #2: **Action Cache (AC)**.
 - Stores mapping of Action → ActionResult. A cache of previously run actions.
 - Small. Can be poisoned. Restrict write access to trusted workers and clients!



The Buildbarn project

A fairly complete Buildbarn setup



Note: not all components are required. Just cherry-pick the parts you need!



bb_storage

Introducing BlobAccess

BlobAccess is Buildbarn's internal uniform API for accessing the Content Addressable Storage and Action Cache.

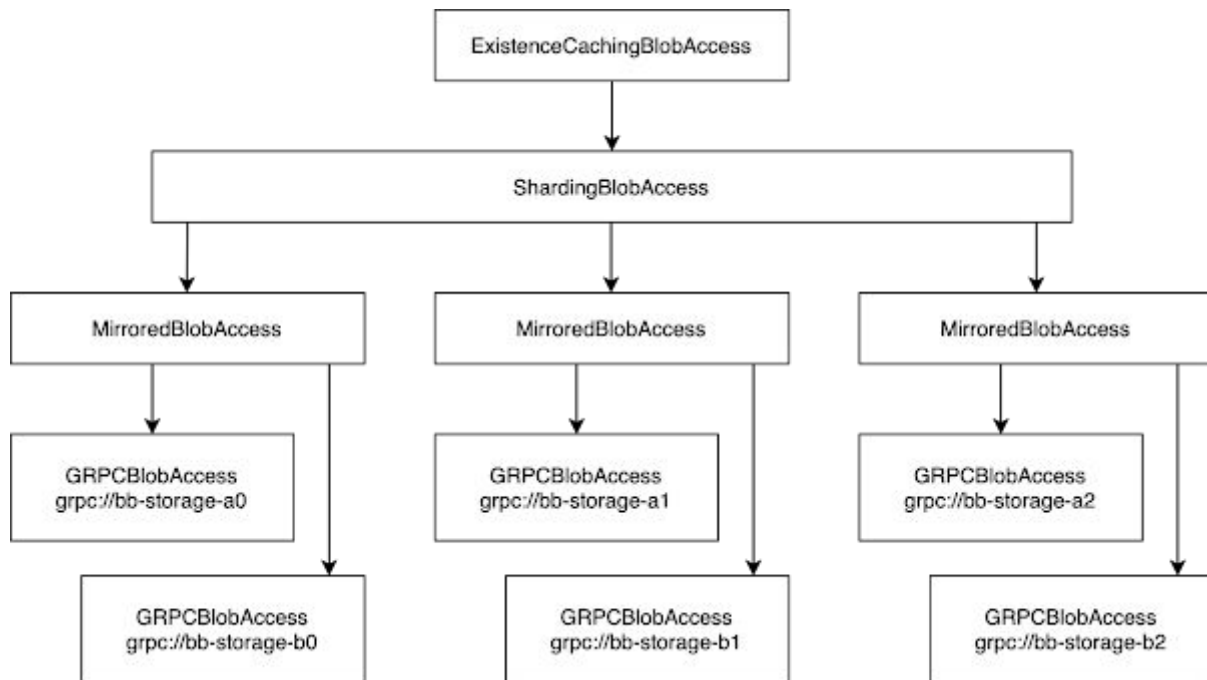
```
type BlobAccess interface {  
    // Read an object from the CAS or AC.  
    Get(ctx context.Context, digest digest.Digest) buffer.Buffer  
    // Write an object to the CAS or AC.  
    Put(ctx context.Context, digest digest.Digest, b buffer.Buffer) error  
    // Express the intent of accessing CAS objects in the nearby future.  
    FindMissing(ctx context.Context, digests digest.Set) (digest.Set, error)  
}
```



Implementations of BlobAccess

- Primitive:
 - **LocalBlobAccess**: store data in memory or on disk.
 - **GRPCBlobAccess**: forward traffic across the network using gRPC.
- Composite:
 - **ReadCachingBlobAccess**: let storage backend A be a cache for backend B.
 - **ShardingBlobAccess**: partition the data set across multiple backends (“RAID-0”).
 - **MirroredBlobAccess**: let two storage backends mirror each other (“RAID-1”).
 - **CompletenessCheckingBlobAccess**: hide AC entries that refer to nonexistent CAS objects.
 - **DemultiplexingBlobAccess**: route traffic to multiple backends based on the request.
 - **ExistenceCachingBlobAccess**: cache positive FindMissing() responses.
 - **MetricsBlobAccess**: expose timing/size stats as Prometheus metrics.
 - **SuspendableBlobAccess**: Suspends a clock, compensating for I/O time.
 - ...

Example 1: “RAID-10” CAS of six storage nodes

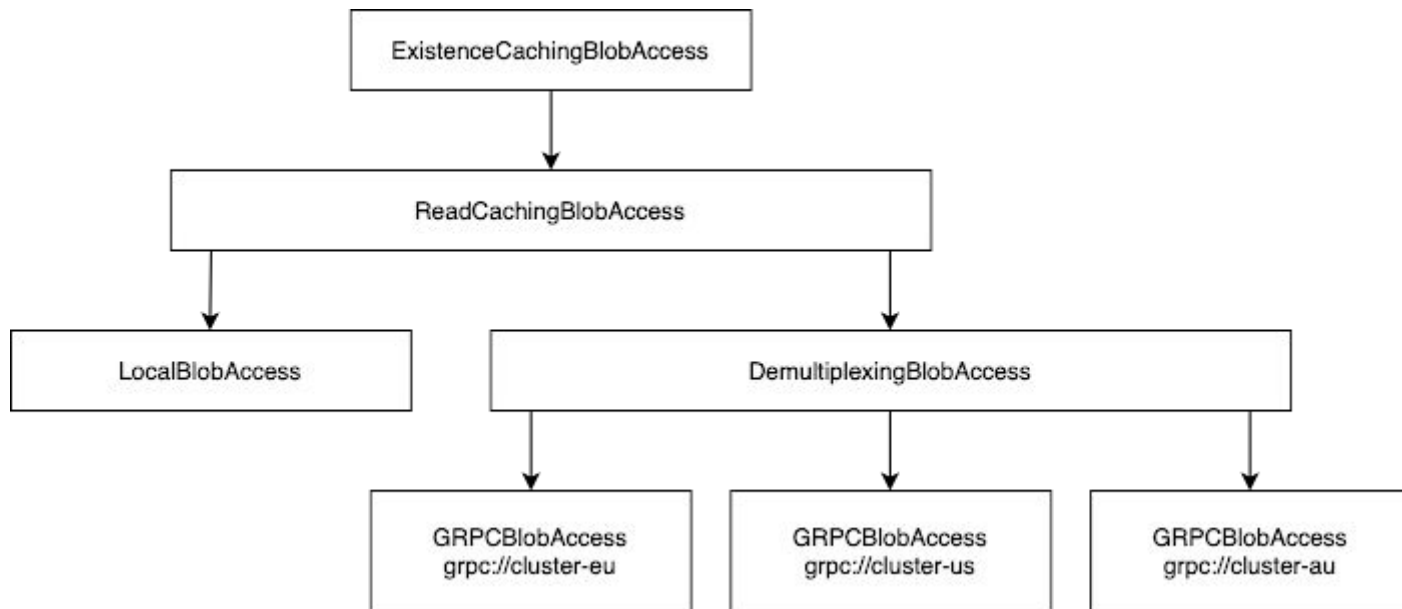


Example 1: “RAID-10” CAS of six storage nodes

```
local casConfig = { existenceCaching: {  
  backend: {  
    sharding: {  
      shards: [  
        { mirrored: {  
          backendA: { grpc: { ... } },  
          backendB: { grpc: { ... } },  
          ...  
        } },  
        ...  
      ],  
    },  
  },  
  ...  
} };
```



Example 2: On-prem cache for multiple clusters



Example 2: On-prem cache for multiple clusters

```
local casConfig = { existenceCaching: {  
  backend: {  
    readCaching: {  
      slow: {  
        demultiplexing: {  
          instanceNamePrefixes: {  
            cluster-eu: { backend: { grpc: { ... } } } },  
            cluster-us: { backend: { grpc: { ... } } } },  
            cluster-au: { backend: { grpc: { ... } } } },  
          }  
        }  
      },  
      fast: { 'local': { ... } },  
    }  
  },  
  ...  
} };
```





bb_storage:
LocalBlobAccess

LocalBlobAccess features

- Custom disk-based storage backend. Not based on third-party library.
- Can store data in memory (fastest), on raw disk (fast), or in a file (slower).
- **Size constrained:** quasi-LRU.
 - Recently accessed blobs are guaranteed to remain present.
 - Blobs that haven't been accessed recently are overwritten.
 - Needed to support Bazel's 'Builds without the Bytes'.
- **Predictable:** Doesn't need any background compactions.
- **Instant restarts:** Doesn't need any scanning/reloading on startup.
- **Self-healing:** Discards data when data corruption is detected.



LocalBlobAccess under the hood

- **Key-location map:**

- Hash table that maps a blob's digest to a location on disk.
- Uses Robin Hood hashing to push older entries deeper into the hash table.
- All entries have seeded checksums, to be able to do bulk invalidations.

- **Location-blob map:**

- Partitions disk into tens of huge blocks.
- Each block contains blobs that are written consecutively.
- Blocks are recycled round-robin.

- **Persistency state:**

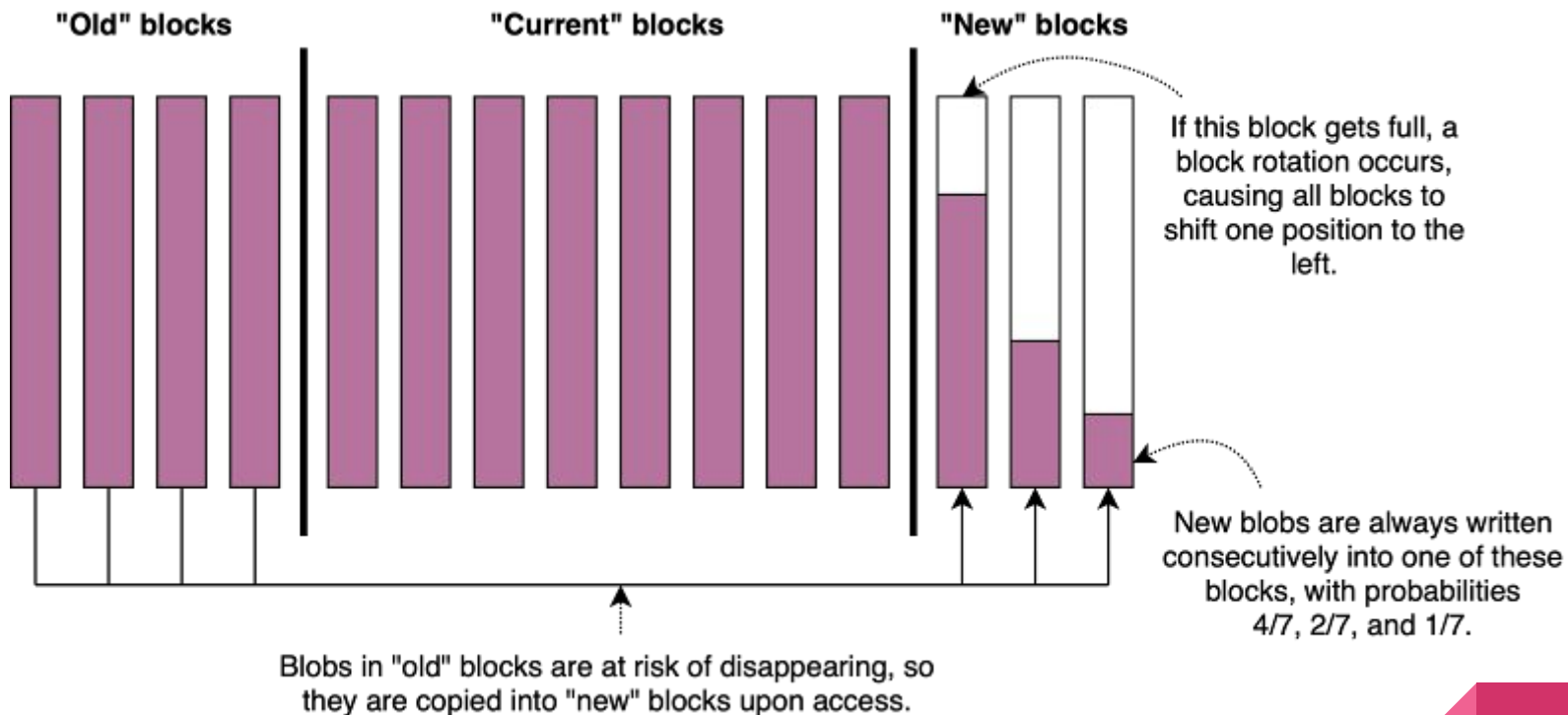
- Small file that stores hash table checksum seeds and layout of blocks on disk.
- Optional. Without it, LocalBlobAccess is empty after every restart.
- Want to clear the cache? Just delete this small file!



LocalBlobAccess class diagram



LocalBlobAccess block handling





bb_storage:
MirroredBlobAccess

MirroredBlobAccess: RAID-1 across servers

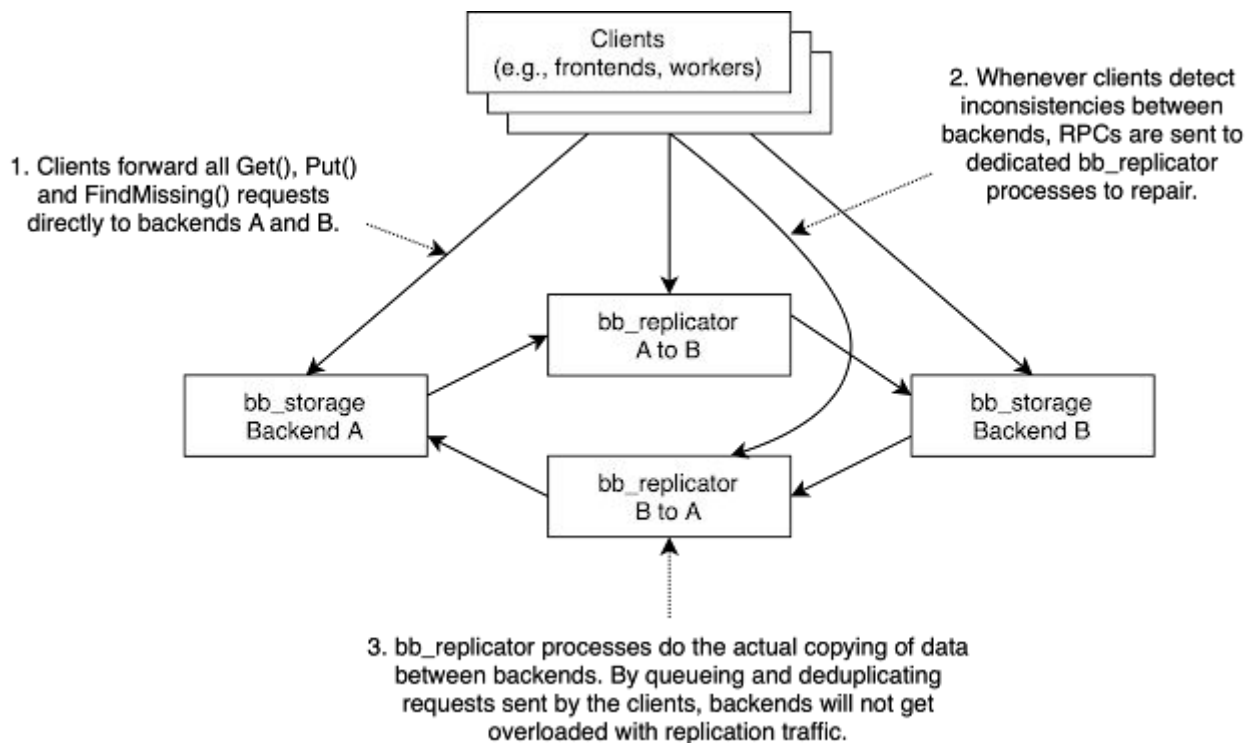
- **Get():** Coin flip: fetch 50% of blobs from replica A, 50% from B.
 - Fall back to other replica if the first replica returns NOT_FOUND.
 - If second replica returns OK: stream data from second replica to first while reading.
- **Put():** Stream blob to both replica A and B.
- **FindMissing():** Forward request to both replica A and B.
 - Return the intersection of results to the client.
 - Copy blobs between replicas based on differences in results.
- **Safe to replace a (broken) replica with an empty one.**
 - Builds can continue, assuming other half doesn't disappear before completion.
 - No logic for proactively syncing data between replicas. It's just a cache!
 - Recreate replica A, wait a week, recreate replica B, wait a week, ...

Cascading failures under high load

- **Problem:** huge load spikes if one replica suddenly loses all data.
 - All clients discover that frequently used blobs are absent.
 - All of them will instantly try to resynchronize these blobs between replicas.
 - Replicas get overloaded, potentially causing them to drop traffic, get OOM killed, etc.
 - Not a problem for the AC: low probability of two clients accessing the same ActionResult.
- **Solution:** control replication traffic centrally.
 - Let all clients call into a dedicated bb_replicator process.
 - bb_replicator can deduplicate incoming replication requests.
 - bb_replicator can apply concurrency limits.



MirroredBlobAccess with dedicated replicators



Introducing BlobReplicator

BlobReplicator is used by MirroredBlobAccess, ReadCachingBlobAccess and ReadFallbackBlobAccess to do the actual replication of blobs.

```
type BlobReplicator interface {  
    // Replicate a single object between backends, while at the same  
    // time giving a handle back to it. Used by Get().  
    ReplicateSingle(ctx context.Context, digest digest.Digest) buffer.Buffer  
    // Replicate a set of objects between backends. Used by FindMissing().  
    ReplicateMultiple(ctx context.Context, digests digest.Set) error  
}
```



Implementations of BlobReplicator

- Primitive:
 - **LocalBlobReplicator**: let the client copy blobs between replicas directly.
 - Good enough to use on its own for the AC.
 - **RemoteBlobReplicator**: forward replication traffic over gRPC to bb_replicator.
 - **NoopBlobReplicator**: don't attempt to repair any inconsistencies.
- Composite:
 - **QueuedBlobReplicator**: deduplicate replication requests, executing them sequentially.
 - Good for distributed setups, such as bb_replicator for a mirrored CAS.
 - **DeduplicatingBlobReplicator**: deduplicate replication requests, don't run them sequentially.
 - Good for local setups, such as ReadCachingBlobAccess on top of LocalBlobAccess.





bb_storage: The Buffer API

The complexity of data handling

- Buildbarn needs to read/write data in various shapes:
 - Byte arrays containing the entire object (e.g., Redis).
 - Streams (e.g., gRPC, HTTP).
 - Random access (e.g., files on disk, reads against FUSE).
 - Protobuf messages (e.g., Action, Command, Directory, ActionResult, Tree)
 - **Sending and receiving side may not use the same shape.**
- Objects can be tiny (<100 B), but also huge (>10 GB).
 - “Just convert everything to a byte array!” is not a valid strategy.
- MirroredBlobAccess needs to multiplex data.
 - Can't perform an infinite amount of buffering; would also exhaust memory.
- **Solution:** Add a proper Buffer API to abstract this away.



Creating Buffers

- Buffers that check that data matches a digest (for the CAS):
 - func **NewCASBufferFromByteSlice**(digest digest.Digest, data []byte, source Source) Buffer
 - func **NewCASBufferFromChunkReader**(digest digest.Digest, r ChunkReader, source Source) Buffer
 - func **NewCASBufferFromReader**(digest digest.Digest, r io.ReadCloser, source Source) Buffer
- Buffers that check that data contains a valid Protobuf message (for the AC):
 - func **NewProtoBufferFromByteSlice**(message proto.Message, data []byte, source Source) Buffer
 - func **NewProtoBufferFromProto**(message proto.Message, source Source) Buffer
 - func **NewProtoBufferFromReader**(message proto.Message, r io.ReadCloser, source Source) Buffer
- Buffers that don't do any data integrity checking:
 - func **NewValidatedBufferFromByteSlice**(data []byte) Buffer
 - func **NewValidatedBufferFromReaderAt**(r ReadAtCloser, sizeBytes int64) Buffer
- Buffers that always return an error:
 - func **NewBufferFromError**(err error) Buffer



Using Buffers

```
type Buffer interface {  
    // Read data from the Buffer, destroying it in the process.  
    IntoWriter(w io.Writer) error  
    ReadAt(p []byte, off int64) (int, error)  
    ToProto(m proto.Message, maximumSizeBytes int) (proto.Message, error)  
    ToByteSlice(maximumSizeBytes int) ([]byte, error)  
    ToChunkReader(off int64, maximumChunkSizeBytes int) ChunkReader  
    ToReader() io.ReadCloser  
  
    // Multiplex the data in the Buffer, either by copying it up front or streaming it.  
    CloneCopy(maximumSizeBytes int) (Buffer, Buffer)  
    CloneStream() (Buffer, Buffer)  
  
    // Don't do anything with the data after all.  
    Discard()  
}
```



Data integrity checking

- Most buffer types have a Source associated with them.
 - **buffer.UserProvided**: Data is being uploaded into Buildbarn.
 - **buffer.BackendProvided**: Data is being downloaded from Buildbarn.
- The Source is used to determine what to do when integrity checks fail.
 - CAS: SHA-256 mismatch. AC: Protobuf unmarshal failure.
 - **buffer.UserProvided**: Could be a workstation with broken memory/disk.
 - Just return INVALID_ARGUMENT to the user. End of story.
 - **buffer.BackendProvided**: Infrastructure failure!
 - Optional DataIntegrityCallback can be used by backends to act on failures.
 - RedisBlobAccess will delete individual keys.
 - LocalBlobAccess will discard all blocks up to and including the corruption.



RedisBlobAccess.Get() simplified

```
func (ba *redisBlobAccess) Get(ctx context.Context, digest digest.Digest) buffer.Buffer {
    value, err := ba.redisClient.Get(ctx, digest.String()).Bytes()
    if err != nil {
        return buffer.NewBufferFromError(err)
    }
    return buffer.NewCASBufferFromByteSlice(
        digest,
        value,
        buffer.BackendProvided(
            func(dataIsValid bool) { // DataIntegrityCallback.
                if !dataIsValid {
                    ba.redisClient.Del(ctx, key)
                }
            }
        )))
}
```



Error handlers

- **ReadCachingBlobAccess** reads data remotely if not present locally.
- **MirroredBlobAccess** reads data from backend B if A doesn't have it.
- **MetricsBlobAccess** increases Prometheus counters based on I/O outcome.
- **SuspendingBlobAccess** delays execution timeouts based on I/O duration.

```
func WithErrorHandler(b Buffer, errorHandler ErrorHandler) Buffer  
  
type ErrorHandler interface {  
    // Continue with a replacement buffer, or return a true failure.  
    OnError(err error) (Buffer, error)  
    Done()  
}
```



ReadCachingBlobAccess.Get()

```
func (ba *readCachingBlobAccess) Get(ctx context.Context, digest digest.Digest) buffer.Buffer {
    return buffer.WithErrorHandler(
        ba.fast.Get(ctx, digest),
        &readCachingErrorHandler{
            replicator: ba.replicator,
            context:    ctx,
            digest:     digest,
        })
}

func (eh *readCachingErrorHandler) OnError(observedErr error) (buffer.Buffer, error) {
    if eh.replicator == nil || status.Code(observedErr) != codes.NotFound {
        return nil, observedErr
    }
    replicator := eh.replicator
    eh.replicator = nil
    return replicator.ReplicateSingle(eh.context, eh.digest), nil
}
```



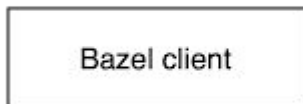


bb_storage:
Indirect Content Addressable
Storage (ICAS)

Remote dependencies

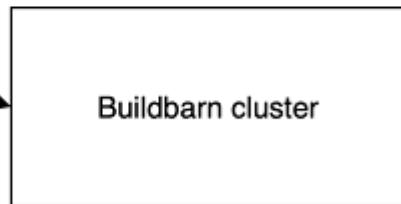
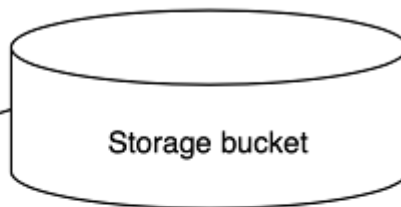
Office or home network

1. Bazel needs to run a test that depends on a large data set, so it downloads it from a storage bucket in the data center.



2. The test is permitted to run remotely, so Bazel uploads it into Buildbarn's CAS.

Data center



Unnecessary cross-site network traffic.

Bad ways of avoiding unnecessary traffic

- Let tests communicate directly with the storage bucket.
 - Adds more boilerplate code to tests.
 - Introduces non-hermeticity. May require you to purge the AC frequently.
 - Poor caching: '--runs_per_test 100' may cause a hundred downloads.
 - Flakiness: network I/O errors count as hard test failures.
- Create an NFS/SMB/... share that is mounted everywhere.
 - Lack of fine-grained dependencies. All tests can access all data in the network share.
 - NFS/SMB security model is different from gRPC. No TLS & JWT.
- Place downloading of data in a separate remote action.
 - Requires the use of 'Builds without the Bytes' (which may be acceptable).
 - Causes dependency to be fully ingested into the CAS, which may be wasteful.

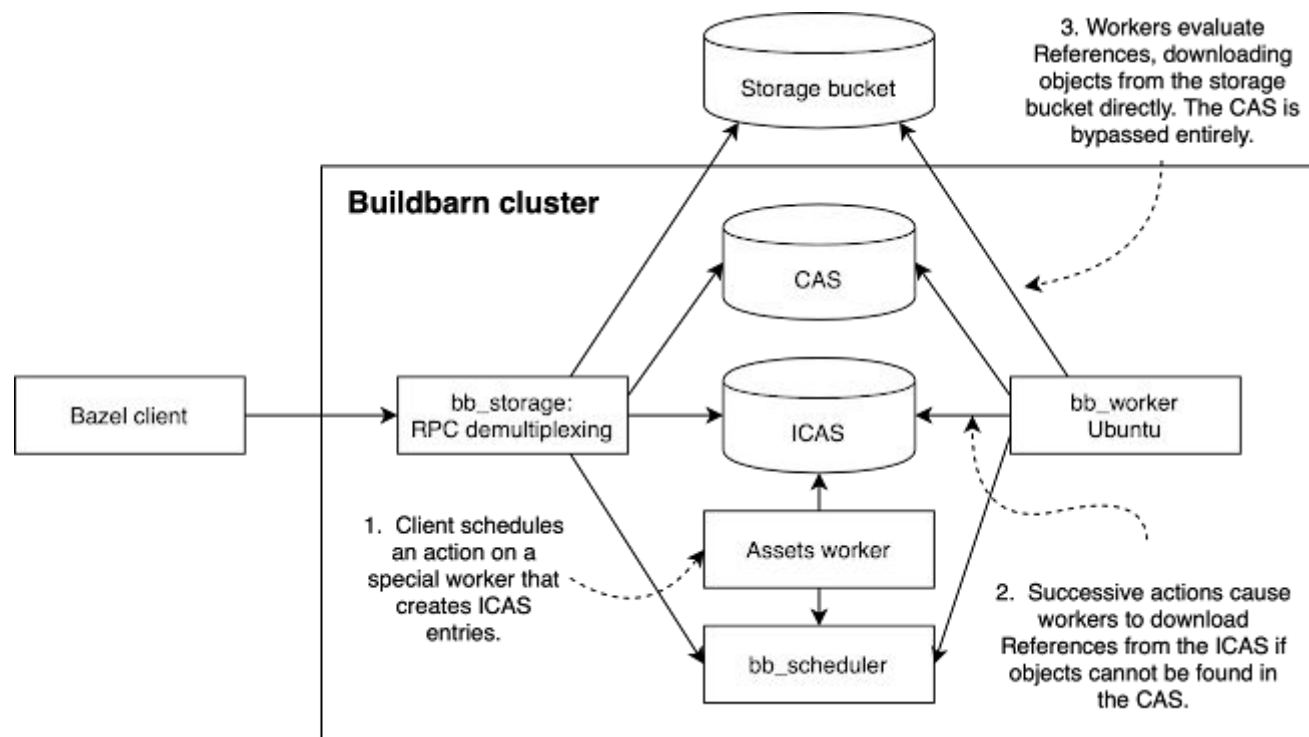


Introducing ICAS

- Basically a CAS, except that it doesn't store any data.
 - It stores Reference messages that describe where the data is actually stored.
- The CAS can be augmented with the contents referenced by ICAS:
 - **ReferenceExpandingBlobAccess** follows References, downloading the target file.
 - **ReadFallbackBlobAccess** can be used to combine CAS and ICAS contents.
 - Contents are not ingested into centralized storage; only into worker-level caches.
- Users need to build their own tools to seed the ICAS:
 - Option 1: custom bb_worker that interprets fictive 'fetch' commands.
 - Option 2: provide an implementation of the [Remote Asset API](#).
 - Likely impossible to standardize, as each environment has its own data sources.



Cluster with ICAS

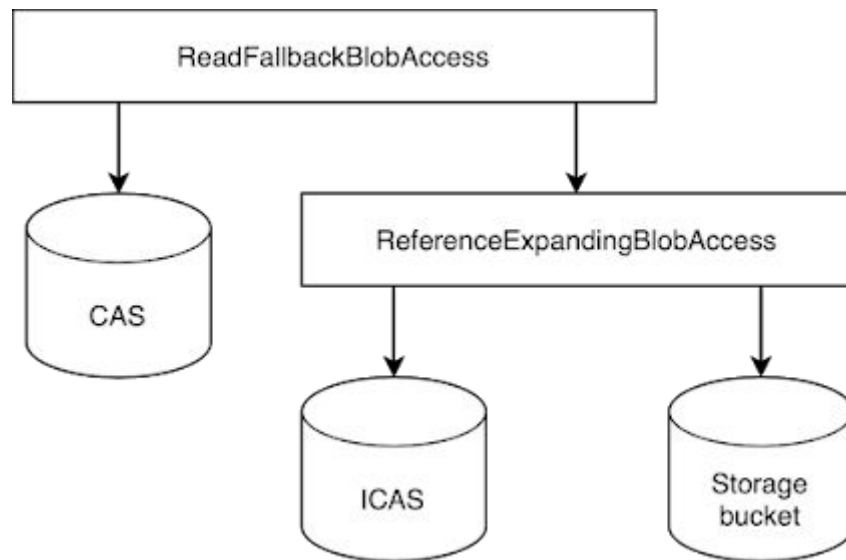


ICAS Reference message

```
message Reference {  
  // HTTP or S3 location where the object may be found.  
  oneof medium {  
    string http_url = 1;  
    S3 s3 = 2;  
  }  
  
  // If the object needs to be extracted from a container format (e.g., a ZIP file).  
  int64 offset_bytes = 3;  
  int64 size_bytes = 4;  
  
  // Transformations that need to be applied when downloading.  
  build.bazel.remote.execution.v2.Compressor.Value decompressor = 6;  
}
```



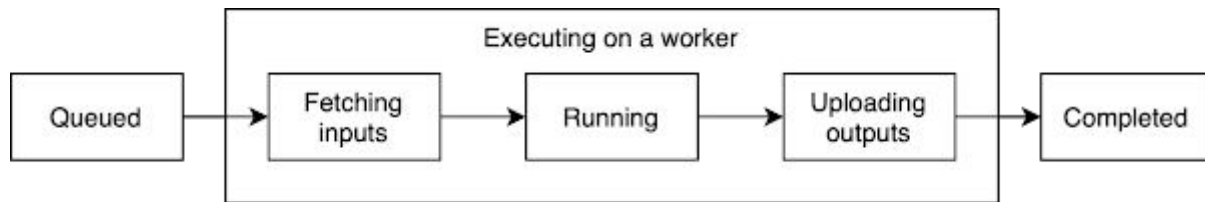
BlobAccess configuration for ICAS





bb_worker:
Runner processes

bb_worker in the past



Monolithic, single-threaded process that repeatedly:

1. Obtains an Action to be run from `bb_scheduler`.
2. Instantiates the input root in the Action message into a local directory.
3. Runs a subprocess in the input root based on the Command message.
4. Uploads the output paths specified in the Command message into the CAS.
5. Typically writes an `ActionResult` into the AC.
6. Returns an `ActionResult` to `bb_scheduler`.

Downsides of the monolithic bb_worker

- Hard to isolate resources.
 - Build action can see the worker process.
 - Kubernetes 'requests' and 'limits' apply to both bb_worker and the build action.
- Hard to parallelize.
 - POSIX bug: multi-threaded processes can't write executables to disk and fork().
 - Result: multi-core servers need to run many bb_workers.
 - Poor CAS cache hit rate, due to lack of cache sharing.
 - Causes many TCP connections to centralized storage and bb_scheduler.
- Not easy to customize.
 - No facilities for running commands in emulators (e.g., QEMU).



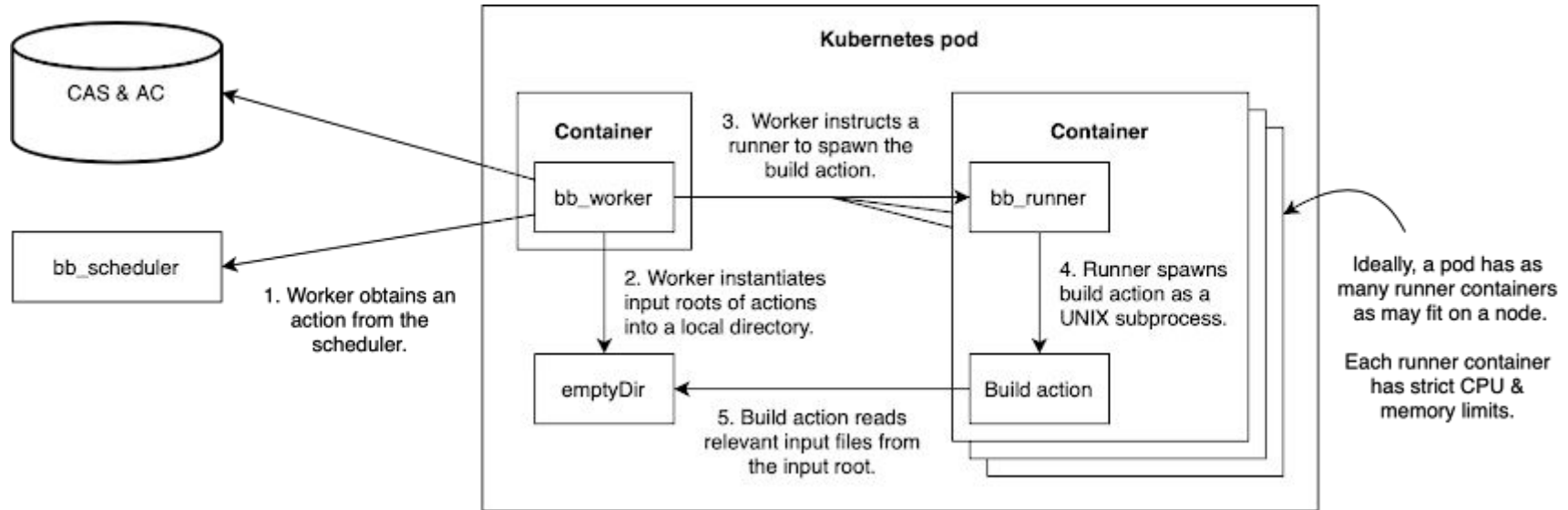
bb_worker in the present

Multi-threaded process, where each thread repeatedly:

1. Obtains an Action to be run from bb_scheduler.
2. Instantiates the input root in the Action message into a local directory.
- ~~3. Runs a subprocess in the input root based on the Command message.~~
Performs a gRPC call into a runner process to launch the subprocess.
4. Uploads the paths specified in the Command message into the CAS.
5. Typically writes an ActionResult into the AC.
6. Returns an ActionResult to bb_scheduler.



A typical worker pod running on Kubernetes



Worker-to-runner protocol (simplified)

```
message RunRequest {  
    repeated string arguments = 1;  
    map<string, string> environment_variables = 2;  
    string working_directory = 3;  
    string stdout_path = 4;  
    string stderr_path = 5;  
}  
  
message RunResponse {  
    int32 exit_code = 1;  
    repeated google.protobuf.Any resource_usage = 2;  
}
```

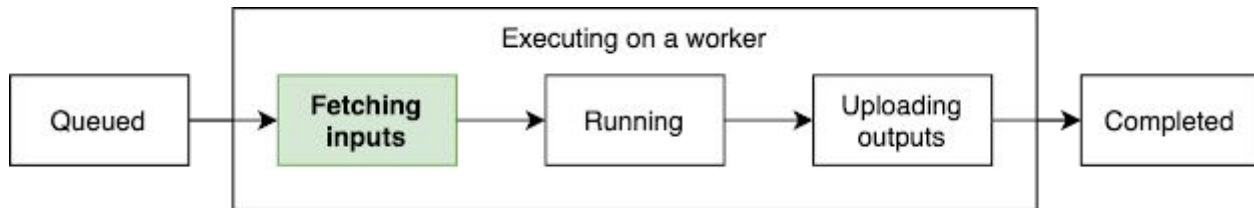




bb_worker:

Input root population

Worker-level file cache



- **Problem:** Input root population can be a costly process.
 - Download gigabytes from the CAS in case actions bring their own SDKs.
 - In case of highly concurrent workers: lots of redundant disk space usage.
- **Solution:** Add a worker-level file cache.
 - A single directory that contains a single copy of every recently used file.
 - **Cache hits:** files are hardlinked from the cache into the input root.
 - **Cache misses:** files are downloaded into the input root and hardlinked into the cache.
 - Population time may become proportional to file count; not total file size.

The contents of the worker-level file cache

```
$ ls -l /worker/cache
```

```
-r-xr-xr-x 1 root root 3259 Jan 1 2000 000095eedb03648c4f9abaf74a248ccbf5a2fb5c0f307ddb31d04bb32cfdcf370-3259+x
-r--r--r-- 1 root root 2469 Jan 1 2000 00010c0e5a6aa2c95ac6b8163176a616fb7e6ff95e93eb040ab4cdb57bdce4d0-2469-x
-r--r--r-- 1 root root 320 Jan 1 2000 000c00bad31d126b054c6ec7f3e02b27c0f9a4d579f987d3c4f879cee1bacb81-320-x
-r-xr-xr-x 1 root root 320 Jan 1 2000 000c00bad31d126b054c6ec7f3e02b27c0f9a4d579f987d3c4f879cee1bacb81-320+x
-r-xr-xr-x 1 root root 742 Jan 1 2000 00145314b959a6dfa16f7d37452f3cf358ef614bdf7b54a28ab9dce9117e31cf-742+x
-r-xr-xr-x 1 root root 1141 Jan 1 2000 001464d2ef94de500cb053cd345164d696f7f84cf38fa522c77327ab04d32982-1141+x
...
-r-xr-xr-x 1 root root 6393 Jan 1 2000 ffc12c3075ba18eaa46dd60e730e06a1ec338216151e056a9c7ecfd74d280fd9-6393+x
-r--r--r-- 1 root root 1913 Jan 1 2000 ffd70c5181898b8b8e17f9f8a76f2d30793abebcd370c739bd1136b34782b09c-1913-x
-r-xr-xr-x 1 root root 3642 Jan 1 2000 ffecdbd3f9ba0d71c6e59984f8384817f0fe5b0ac69ba62e1e40a31faf596a6c-3642+x
-r--r--r-- 1 root root 1440 Jan 1 2000 ffede6eabd976f361148ae9c7e4c7c31bba51db02493f05015d2145db1a4ea44-1440-x
-r--r--r-- 1 root root 3541 Jan 1 2000 ffeeb99fa19cac2db45fa444da20f2c5aa157b87b836bee1f91e18ddf8412c94-3541-x
-r-xr-xr-x 1 root root 75928 Jan 1 2000 fffe9f6f1470c3a12c6588e26f37485b7fac3af83a6f3d52e2205d02d4a646bc-75928+x
```

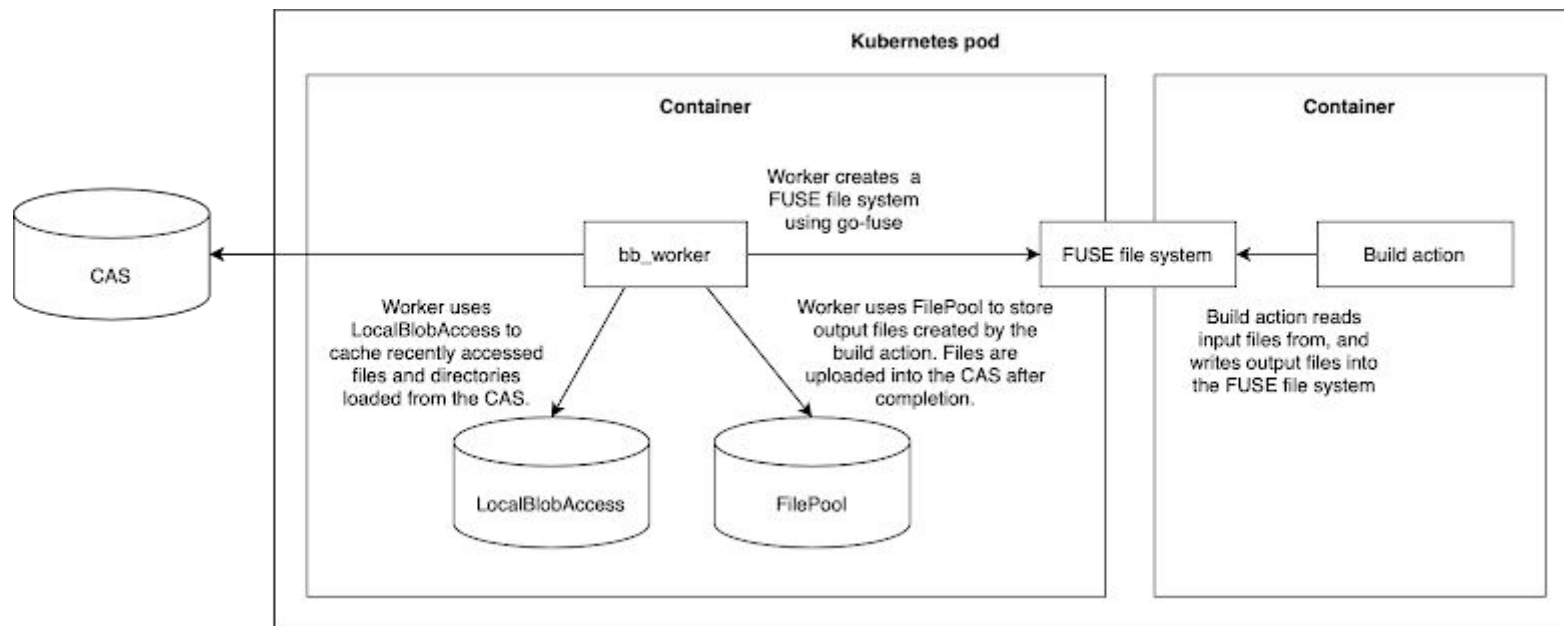
```
$ sha256sum < f7fec7e9c5fec8265163391bb32716ac97deec644637020600869ae3f5cef793-10011+x
f7fec7e9c5fec8265163391bb32716ac97deec644637020600869ae3f5cef793 -
```

Lazy input root population

- **Problem:** Worker-level file cache may still perform poorly.
 - Input roots are downloaded up front entirely, even if only a fraction is used.
 - Requires all runners to share one file system: running out of space causes flakiness for all.
- **Solution:** Run builds inside a virtual file system offered by bb_worker.
 - Starts build actions inside a placeholder input root directory.
 - **Accessing a placeholder directory:** instantiate it, creating placeholder files inside.
 - **Accessing a placeholder file:** read it from the CAS.
 - CAS read errors causes action to be killed, returning a retrieable error.
 - Build actions can create output files inside the input root directory.
 - Quotas are only applied against output files.
 - Exceeding quota causes action to be killed, returning a non-retrieable error.



bb_worker with the virtual file system enabled



Introducing FilePool

FilePool is a general purpose allocator for nameless temporary files. Files created by actions through the virtual file system are backed by them.

```
type FilePool interface {  
    NewFile() (FileReader, error)  
}  
  
type FileReader interface {  
    ReadAt(b []byte, off int64) (int, error)  
    WriteAt(b []byte, off int64) (int, error)  
    Close() error  
}
```



Implementations of FilePool

- Primitive:
 - **InMemoryFilePool**: store all data in memory.
 - **BlockDeviceBackedFilePool**: store all files on a raw disk (or in a fixed-size file).
 - **DirectoryBackedFilePool**: store all files in a single directory, using integer names (slow).
- Composite:
 - **QuotaEnforcingFilePool**: wraps an existing FilePool, adding quota limits.
 - Every action gets its own instance, all wrapping the same underlying FilePool.
 - **StatsCollectingFilePool**: measures total & peak resource usage.
 - Stats are logged as part of ActionResult metadata.

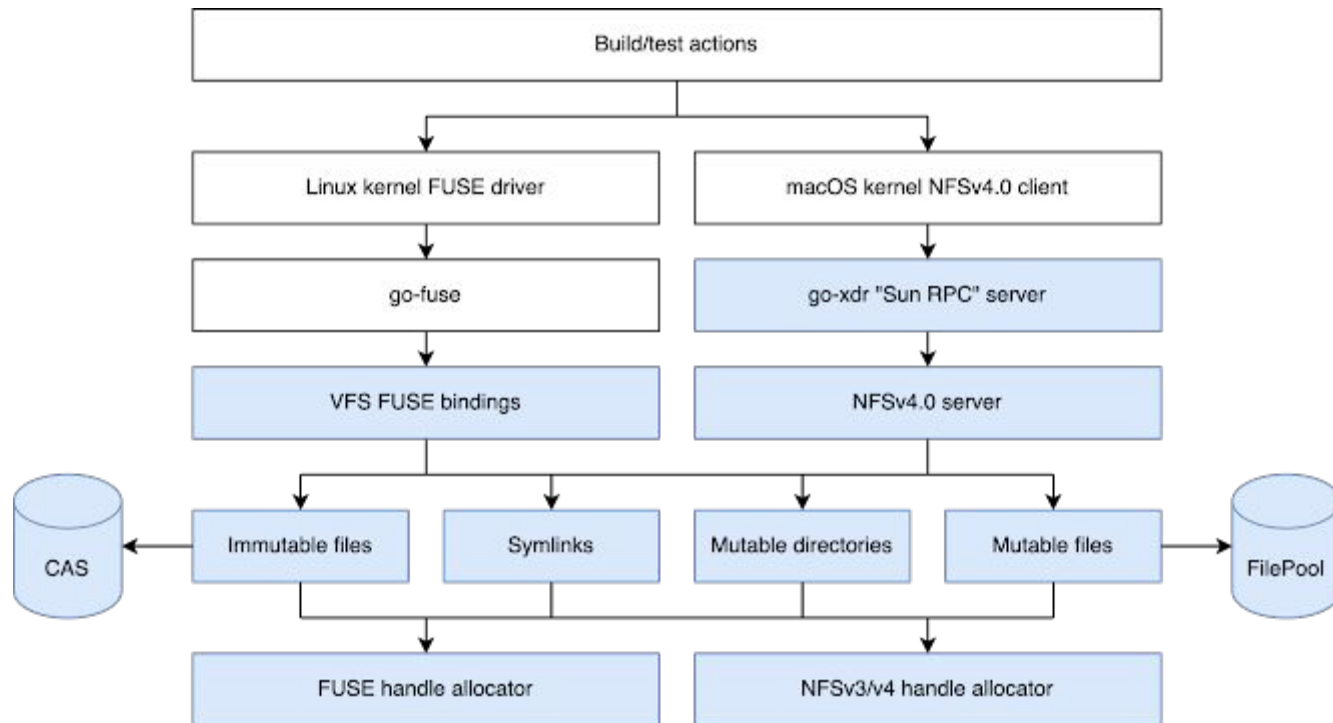


Recent developments in the virtual file system

- 2020: Initial release of the FUSE file system for bb_worker.
 - Linux: use in-kernel FUSE driver, which works great.
 - macOS: use OSXFUSE, which isn't always stable (and proprietary).
- 2021: Read delays compensate execution timeout.
 - Adds notion of 'virtual execution duration' to REv2.
- 2022: FUSE file system refactored into a generic virtual file system.
 - Added support for NFSv4.0 in addition to FUSE.
 - macOS 13.3+ systems can 'mount -t nfs localhost' the integrated NFSv4.0 server.
- 2023: File system access profiling.
 - bb_worker can record which paths are accessed during build, and store those centrally.
 - Subsequent executions of similar actions reload the profile from central storage.
 - Profile is used to load expected set of accessed files in the background, in parallel.



Layering of the virtual file system



Note: components in blue are maintained as part of the Buildbarn project



bb_scheduler

bb_scheduler overview

- Multiple levels of nested queues.
 - Scheduler has one platform queue for every distinct platform (Ubuntu, CentOS, macOS, etc.)
 - Every platform queue has one or more size class queues.
 - Every size class queue has a queue of running Bazel invocations.
 - Every Bazel invocation has a queue of scheduled operations (actions).
 - **Scheduler steers towards fairness between Bazel invocations.**
- Scheduler keeps all queue state in memory; no persistency.
 - Restart means that clients need to restart all current operations.
 - Not an issue, considering that scheduler restarts don't happen frequently.
- gRPC protocol and integrated web UI for exploring the current worker state.
- Uses a simple and robust scheduler ↔ worker protocol.
 - Specific to Buildbarn. No consensus within community yet.

Screenshot of the bb_scheduler web UI

Buildbarn Scheduler

Build queue

Total number of operations: 6010

Platform queues

Instance name prefix	Platform properties	Size class	Timeout	Queued operations	Root invocation							All workers	Drains
					Children			Workers					
					Queued	Active	All	Executing	Idle	synchronizing			
...	arch="x86_64" os="centos" os_version="7"	0	∞	0	0	2	3	46	754	105	800	0	
...	arch="x86_64" os="centos" os_version="8"	0	∞	0	0	0	0	0	80	80	80	0	
...	arch="x86_64" os="ubuntu" os_version="bionic"	0	∞	0	0	0	0	0	160	160	160	0	
...	arch="x86_64" os="ubuntu" os_version="jammy"	8	∞	0	0	0	0	0	10	10	10	0	

Worker-scheduler state synchronization

```
message SynchronizeRequest { // What the worker sends to the scheduler.
    // Freeform labels that identify a worker: {"dc": "paris", "rack": "5", "slot": "3"}.
    // Every worker thread must have a unique set of labels.
    map<string, string> worker_id = 1;
    // Identifier of the size class queue (which actions to pick up).
    string instance_name = 2;
    build.bazel.remote.execution.v2.Platform platform = 3;
    uint32 size_class = 5;
    // Current state of the worker. Is an action running? If so, which one?
    CurrentState current_state = 4;
}

message SynchronizeResponse { // What the scheduler sends in response.
    // "Please don't contact me until [...], unless the current action is completed."
    google.protobuf.Timestamp next_synchronization_at = 1;
    // What the scheduler wants the worker to do: run a new action, go idle, or continue.
    DesiredState desired_state = 2;
}
```





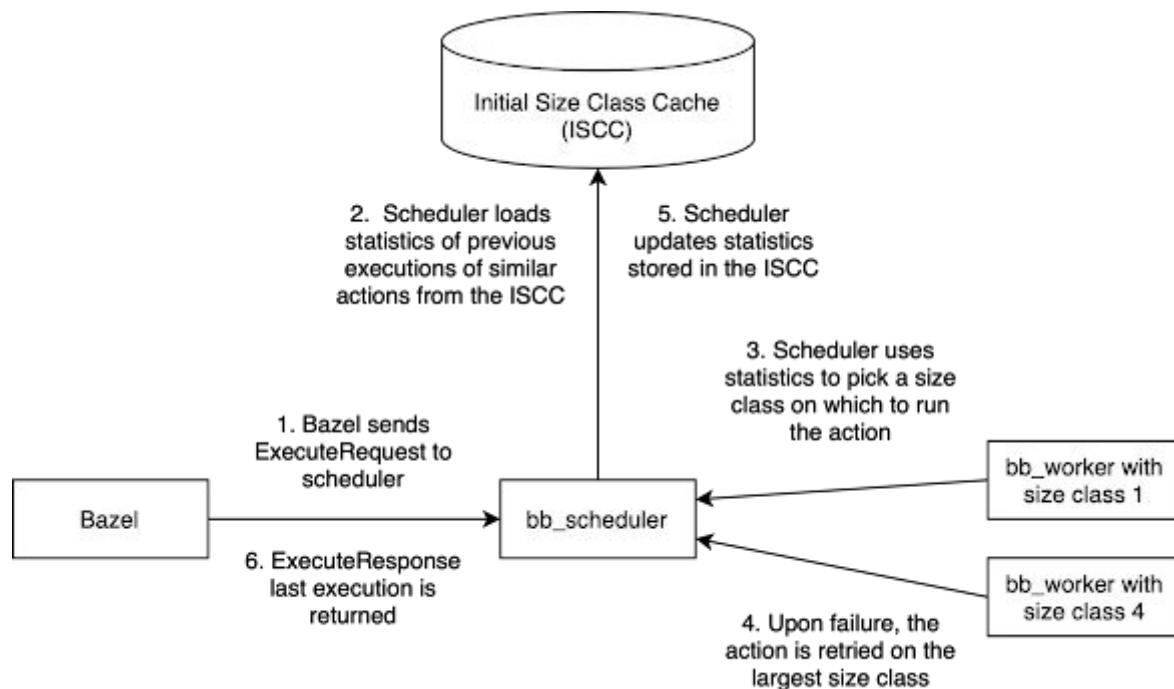
bb_scheduler:
Worker size classes

Problem definition

- Resource requirements of actions follow a hockey stick curve.
 - Almost all of them are single-threaded / don't use a lot of RAM.
 - Only a handful benefit from multi-threaded workers / need more RAM.
- **Traditional solution:** Multiple worker types with different platform properties.
 - Requires annotating targets in BUILD files.
 - Race to the bottom: "I just want my test to run fast. Let me pick the largest worker!"
 - Hard to get right when multiple build configurations are used (e.g., release, debug, asan).
 - Bazel lacks granularity for doing this right (e.g., when using sharded tests).
 - Third-party libraries that ship with BUILD files cannot get it right for your cluster.
 - Annotations need to be revisited every time infrastructure is redesigned.
- **Desired solution:** Let Buildbarn sort it out itself.



Rough outline of the solution



Note: arrows indicate directions of RPCs

When is it acceptable to use smaller workers?

Not only do they need to succeed. The action also needs to run fast enough.

- Assume there is an action that runs in 10 seconds on size class 4.
 - Expected execution time of 11 seconds on size class 1: acceptable.
 - Expected execution time of 72 seconds on size class 1: unacceptable.
- Scheduler computes cutoff with $\text{median}(T_{\text{largest}}) \cdot (SC_{\text{largest}} / SC_{\text{smaller}})^k$
 - k is configurable, and must be between (0.0, 1.0).
 - When $k = 0.3$, the cutoff is $10\text{s} \cdot (4/1)^{0.3} = 15.2\text{s}$.
 - When $k = 0.7$, the cutoff is $10\text{s} \cdot (4/1)^{0.7} = 26.4\text{s}$.
 - **k determines 'how parallel' actions must be to prefer execution on larger workers.**
- Runs on smaller workers will use execution timeout based on cutoff.
 - Prevents long stalls in case of mispredictions.

Stats in ISCC for a single kind of action

```
message PreviousExecution {  
  oneof outcome {  
    google.protobuf.Empty failed = 1;  
    google.protobuf.Duration timed_out = 2;  
    google.protobuf.Duration succeeded = 3;  
  }  
}  
  
message PerSizeClassStats {  
  // List of recent outcomes sorted by date, truncated to a configurable size.  
  // Only outcomes of actions that eventually succeeded are stored.  
  repeated PreviousExecution previous_executions = 1;  
  ...  
}  
  
message PreviousExecutionStats {  
  map<uint32, PerSizeClassStats> size_classes = 1;  
  ...  
}
```



Classifying kinds of actions

We need to make sure that the ISCC is keyed by a kind of action with certain performance characteristics.

- Acceptable heuristic: platform properties, argv and environment variables.
 - REv2.1 and lower: Command digest would be perfect.
 - REv2.2 and higher: platform properties have been promoted from Command to Action.
- Including RequestMetadata's configuration_id would be nice.
 - More accurate distinguishing between release and debug builds.
 - Downside: requires that replay tools provide the original metadata.
- ISCC is keyed by a reduced Action digest.
 - Action, but with everything except platform properties and Command digest removed.



Selecting an initial size class with 2 size classes

1. Normalize execution times between size classes using exponent k .
2. Compute the probability of one size class being better/faster than the other.
 - Quadratic component:
 - Mann-Whitney U test. Works great, even if outcomes contain timeouts and failures.
 - Linear component:
 - Give the size class with the fewest samples an advantage, to force training.
 - Constant component:
 - Ensures that empty sets give probability 0.5. Prevents division by zero.
 - Never returns values too close to 0.0 or 1.0, meaning we keep recalibrating.
 - Asymmetric relation: $IsFaster(a, b) + IsFaster(b, a) = 1.0$.
3. Compare probability against a random number to pick the initial size class.



Selecting an initial size class with n size classes

1. Assuming $n = 4$, create a stochastic matrix M with the following structure:

$$\begin{bmatrix} 1.0 - \dots & \text{IsFaster}(a, b) / 3 & \text{IsFaster}(a, c) / 3 & \text{IsFaster}(a, d) / 3 \\ \text{IsFaster}(b, a) / 3 & 1.0 - \dots & \text{IsFaster}(b, c) / 3 & \text{IsFaster}(b, d) / 3 \\ \text{IsFaster}(c, a) / 3 & \text{IsFaster}(c, b) / 3 & 1.0 - \dots & \text{IsFaster}(c, d) / 3 \\ \text{IsFaster}(d, a) / 3 & \text{IsFaster}(d, b) / 3 & \text{IsFaster}(d, c) / 3 & 1.0 - \dots \end{bmatrix}$$

2. Compute eigenvector of matrix M to obtain probability distribution.
3. Compare probabilities against a random number to pick the initial size class.

Quite similar to how [PageRank](#) works.

Tricks to ensure builds still run fast

- Failing actions take longer, as they need to rerun on the largest size class.
 - To distinguish between hard failures and ones caused by lack of resources.
 - Causes slowdowns for iterative at-desk builds (e.g., fixing compiler errors).
 - **Solution:** Store which actions failed recently, and always run them on the largest workers.
- Poor critical path duration if long-running tests fail on too small workers.
 - If there's a 0.1% chance of this per test, there's a 63% chance of this per 1000 tests.
 - Unavoidable; even these tests need their stats recalibrated.
 - **Solution:** Invert the execution strategy on small workers for actions known to fail.
 - First run them on the largest worker and return the results to the client.
 - Rerun them in the background on the smaller worker.
 - Scheduler applies fairness, so that background executions don't dominate.



bb_scheduler web UI with size classes

Buildbarn Scheduler

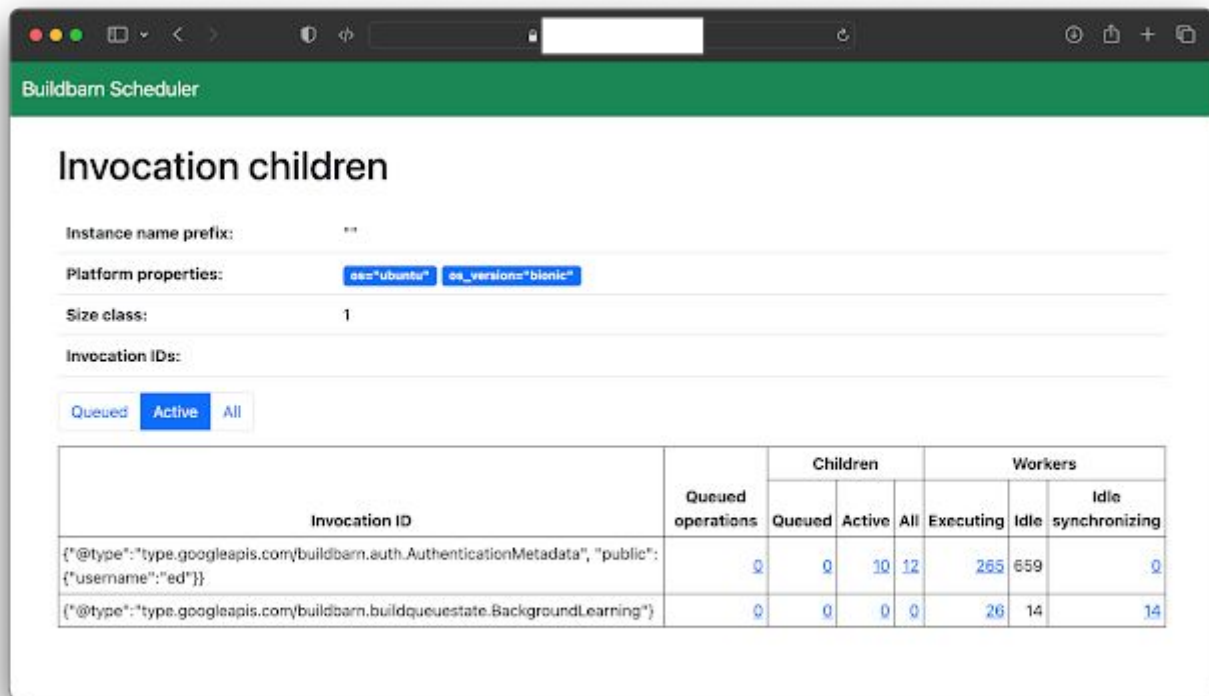
Build queue

Total number of operations: 3381

Platform queues

Instance name prefix	Platform properties	Size class	Timeout	Root invocation							All workers	Drains
				Queued operations	Children			Workers				
					Queued	Active	All	Executing	Idle	Idle synchronizing		
11.0	os="ubuntu" os_version="bionic"	1	∞	0	0	3	4	604	996	382	1600	0
		2	∞	0	0	3	4	362	758	319	1120	0
		4	∞	0	0	2	3	220	680	219	900	0
		8	∞	0	0	2	3	129	71	0	200	0

bb_scheduler web UI with size classes



The screenshot shows the Buildbarn Scheduler web UI. The title bar is green and says "Buildbarn Scheduler". The main heading is "Invocation children". Below this, there are several input fields: "Instance name prefix:" with a text input, "Platform properties:" with two buttons labeled "os='ubuntu'" and "os_version='bionic'", "Size class:" with a text input containing "1", and "Invocation IDs:" with a text input. Below these fields are three tabs: "Queued", "Active", and "All". The "Active" tab is selected. Below the tabs is a table with the following structure:

Invocation ID	Queued operations	Children			Workers		
		Queued	Active	All	Executing	Idle	Idle synchronizing
{"@type":"type.googleapis.com/buildbarn.auth.AuthenticationMetadata", "public": {"username":"ed"}}	0	0	10	12	255	659	0
{"@type":"type.googleapis.com/buildbarn.buildqueuestate.BackgroundLearning"}	0	0	0	0	26	14	14



bb_browser

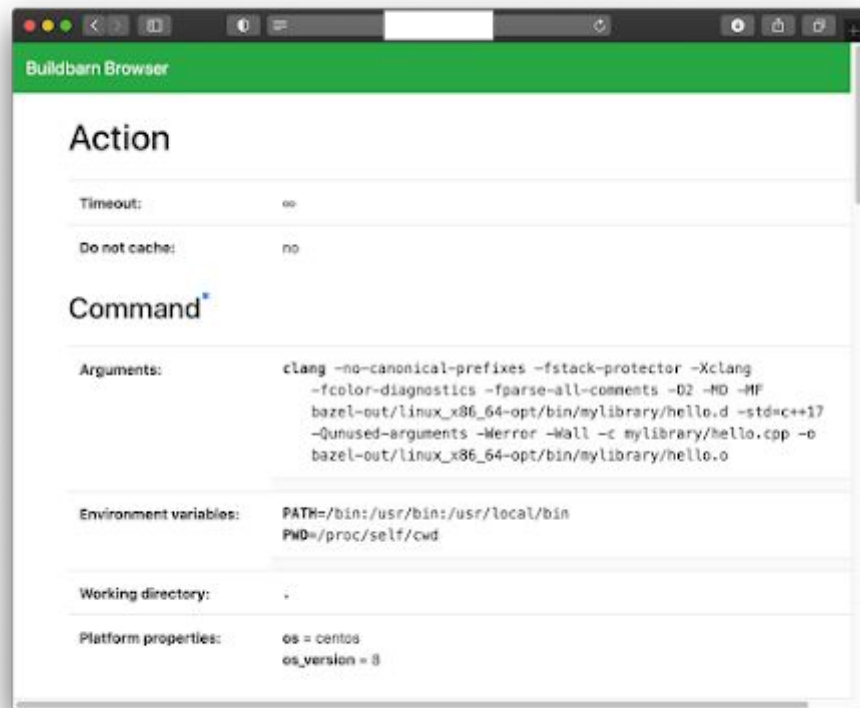
What is bb_browser?

Simple web UI for REv2 data stores.

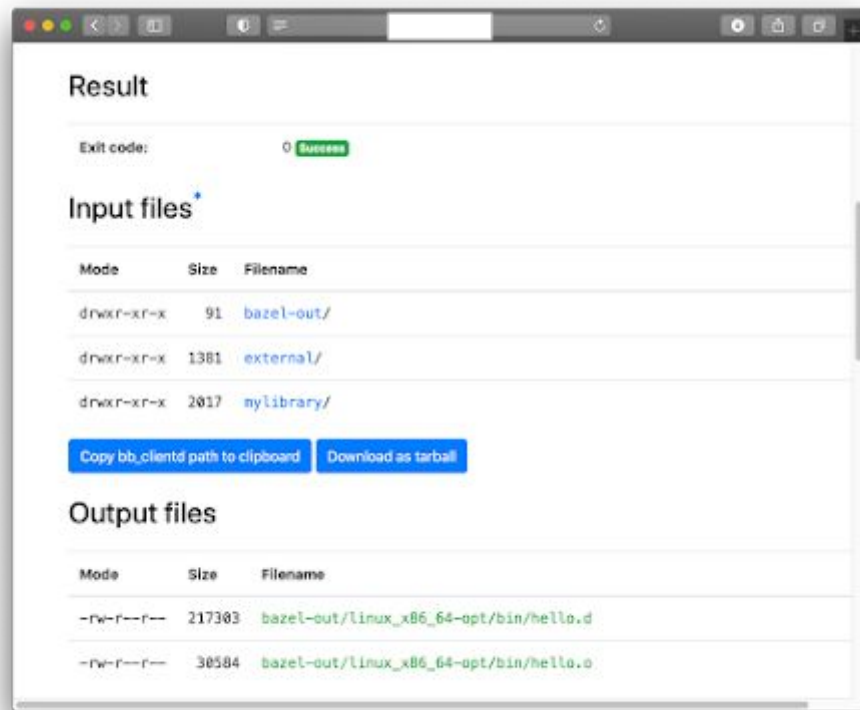
- Can display REv2 Actions, Commands, Directories, ActionResults and Trees.
- Can display Buildbarn specific extensions:
 - UncachedActionResult: Stable copy of a historical ActionResult that's preserved in the CAS.
 - PreviousExecutionStats: Build action timing information that's stored in the ISCC.
 - FileSystemAccessProfiles: Which input files are expected to be used.
- Other Buildbarn tools link to bb_browser.
 - Every operation in bb_scheduler links to an Action page.
 - bb_worker can let Bazel print UncachedActionResult links upon build failures.



Screenshots of bb_browser



Screenshots of bb_browser



Screenshots of bb_browser



The screenshot shows a web browser window with a dark title bar. The page content is divided into two main sections: "Execution metadata" and "Request metadata".

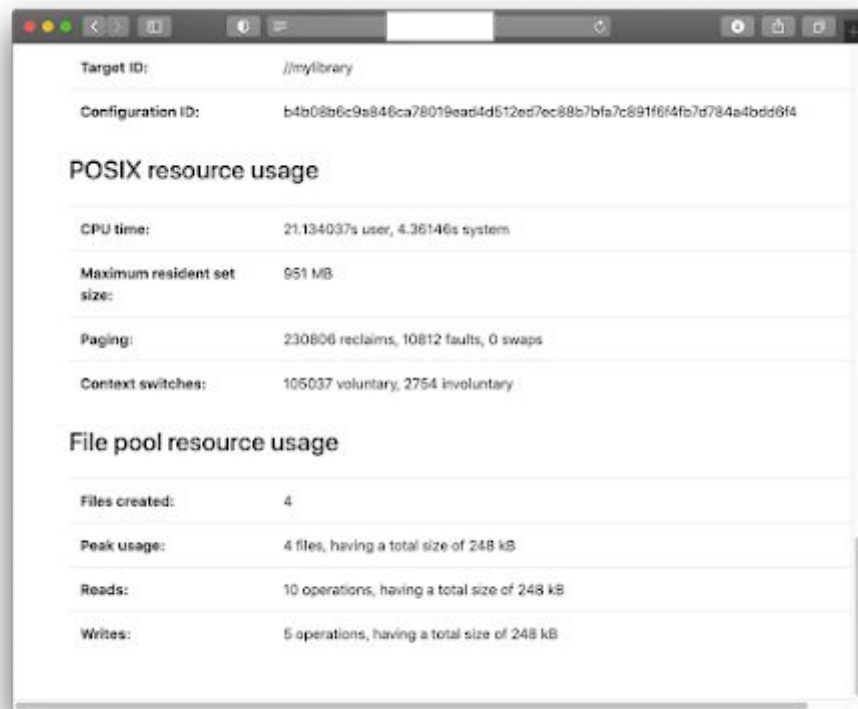
Execution metadata

Worker:	("pod":"centos-8-7tv2s","thread":"3")
Timeline:	2021-04-27T20:14:53.751Z Action added to the queue. 2021-04-27T20:14:53.758Z ⌚ +6.728033ms Worker received the action. 2021-04-27T20:14:53.758Z ⌚ +101µs Worker started fetching action inputs. 2021-04-27T20:14:53.766Z ⌚ +8.461ms Worker finished fetching action inputs. Worker started executing the action command. 2021-04-27T20:15:25.098Z ⌚ +31.331623s Worker completed executing the action command. Worker started uploading action outputs. 2021-04-27T20:15:25.156Z ⌚ +67.527ms Worker completed uploading action outputs. Worker completed the action, including all stages.

Request metadata

Tool:	bazel 4.0.0
Tool invocation ID:	c2e38708-707d-48d5-b5c3-60ff73dca963
Correlated invocations ID:	b150798c-78ab-4256-bad7-3352bb819ae7

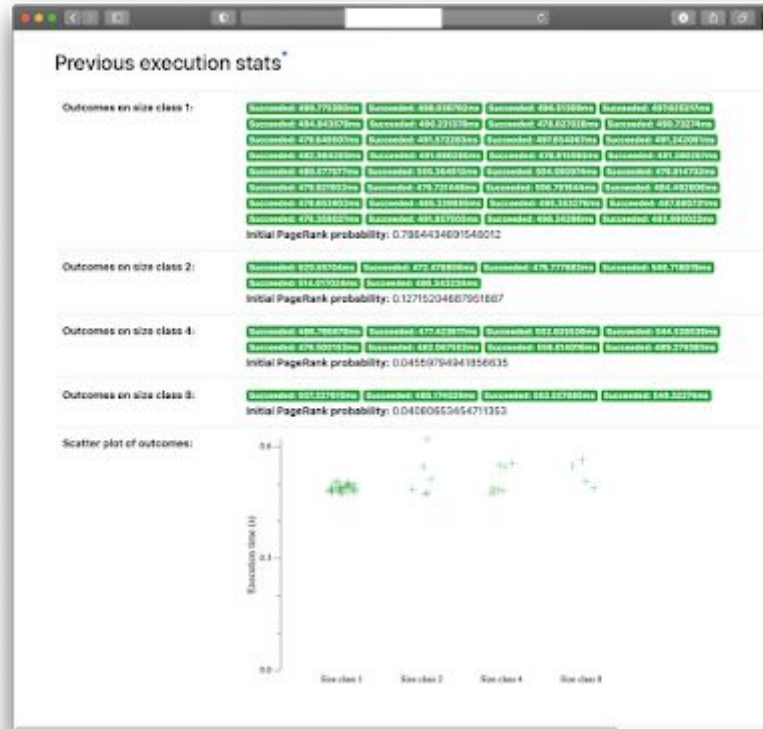
Screenshots of bb_browser



The screenshot shows a window titled 'bb_browser' with a dark title bar. The main content area displays resource usage statistics for a target with ID '/jmylibrary'. The statistics are organized into two sections: 'POSIX resource usage' and 'File pool resource usage'. Each section contains a table of metrics.

Target ID:	/jmylibrary
Configuration ID:	b4b08b6c9s846ca78019ead4d612ed7ec88b7bfa7c891f6f4fb7d784a4bdd6f4
POSIX resource usage	
CPU time:	21.134037s user, 4.36146s system
Maximum resident set size:	951 MB
Paging:	230806 reclaims, 10812 faults, 0 swaps
Context switches:	105037 voluntary, 2754 involuntary
File pool resource usage	
Files created:	4
Peak usage:	4 files, having a total size of 248 kB
Reads:	10 operations, having a total size of 248 kB
Writes:	5 operations, having a total size of 248 kB

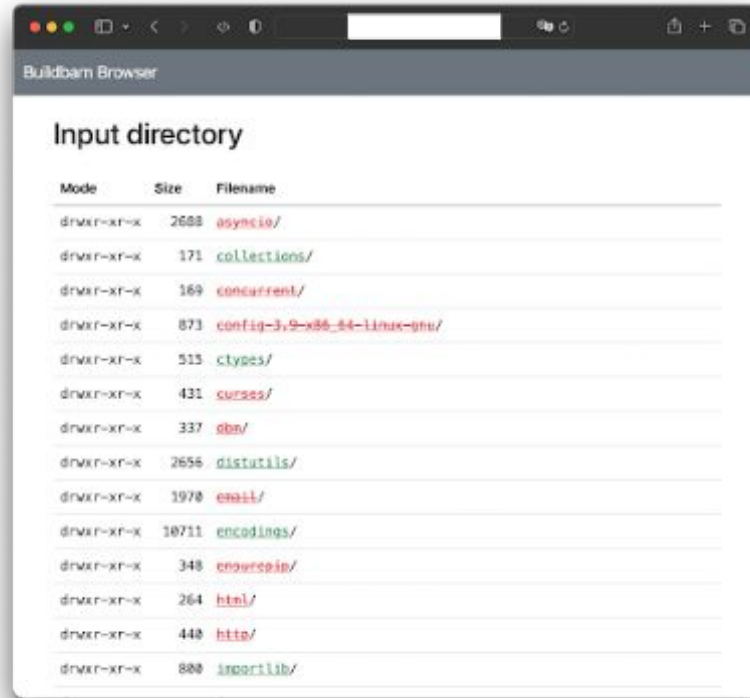
Screenshots of bb_browser



Screenshots of bb_browser



Screenshots of bb_browser





bb_clientd

Bazel's execution modes

	Highly parallel builds	Reproducible execution environment	Multi-user caching	Network efficiency	Low local disk space usage	Ability to access results	Reliable incremental builds
Local execution	✗	✗	✗	✓	✗	✓	✓
Remote caching	✗	✗	✓	✗	✗	✓	✓
Remote execution	✓	✓	✓	✗	✗	✓	✓
Remote execution + Builds w/o the Bytes	✓	✓	✓	✓	✓	✗	✗
What we want	✓	✓	✓	✓	✓	✓	✓

Note: in reality, this may be more nuanced!

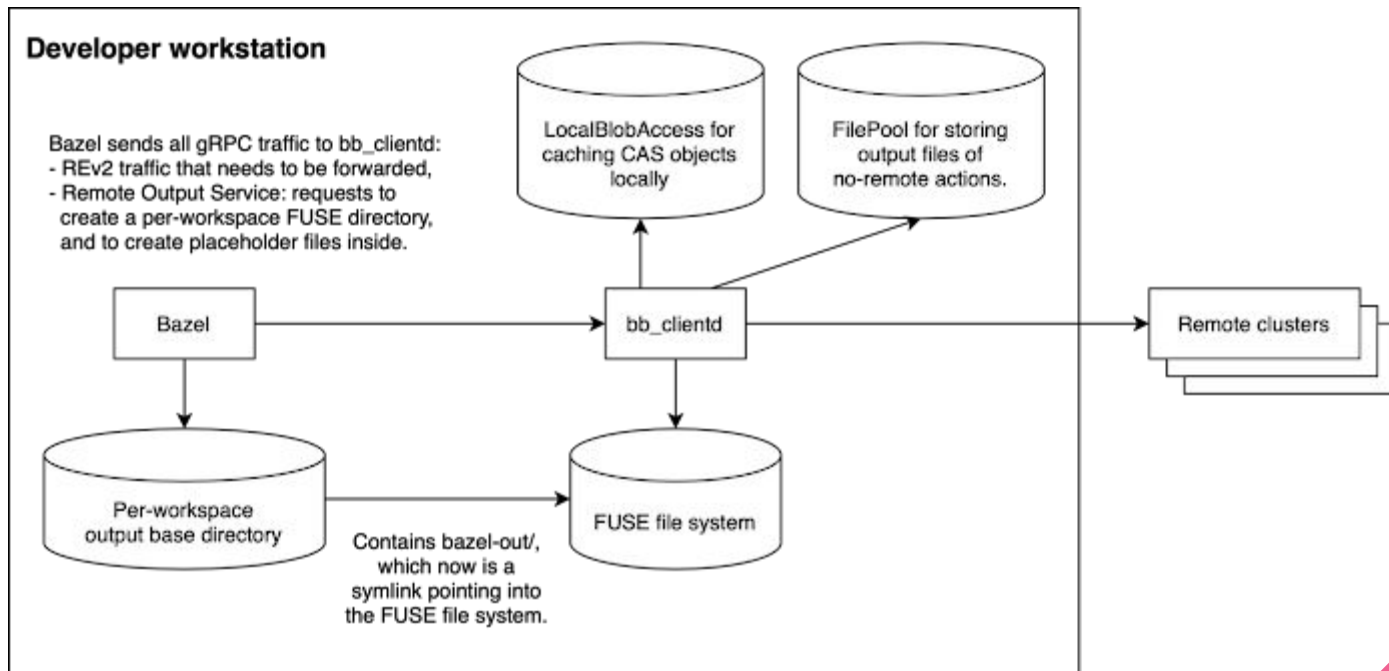
Introducing bb_clientd

bb_clientd is a daemon that you run on your personal system.

- Provides a gRPC server that proxies REv2 traffic to remote clusters.
 - **Prevents redundant Read() traffic** by adding caching using LocalBlobAccess.
 - **Prevents redundant FindMissingBlobs() traffic** by briefly caching server responses.
- Provides FUSE file system that Bazel may use to host the bazel-out/ directory.
 - **Allows Bazel to create remote output files as placeholders**, instead of downloading them.
 - File contents are only downloaded if a locally running action depends on them.
 - Files that disappear from the cluster are automatically pruned at the start of every build.
 - The same performance characteristics as “Builds without the Bytes”.
- Also useful as a general purpose REv2 debugging tool.
 - Can be used to access arbitrary CAS contents.



How Bazel interacts with bb_clientd



gRPC protocol to manage output paths

```
service RemoteOutputService {  
    // Create a directory for the workspace inside the FUSE filesystem.  
    // Path is returned to Bazel, so it can replace bazel-out/ with a symlink to it.  
    // Called at the start of a “bazel build”.  
    rpc StartBuild(StartBuildRequest) returns (StartBuildResponse);  
    // Create one or more placeholder output files for an action that ran remotely.  
    // Called during a “bazel build”.  
    rpc BatchCreate(BatchCreateRequest) returns (google.protobuf.Empty);  
    // Re-obtain the hash and size of one or more files in the FUSE file system.  
    // Calling this on placeholder files does not cause any network I/O.  
    // Called during a “bazel build”.  
    rpc BatchStat(BatchStatRequest) returns (BatchStatResponse);  
    // Called at the end of a “bazel build”.  
    rpc FinalizeBuild(FinalizeBuildRequest) returns (google.protobuf.Empty);  
    // Remove all of the data in the FUSE filesystem corresponding to a given workspace.  
    // Faster than deleting files by hand.  
    // Called when doing a “bazel clean”.  
    rpc Clean(CleanRequest) returns (google.protobuf.Empty);  
}
```



Integration with bb_browser to aid debugging

Input files*

Mode	Size	Filename
drwxr-xr-x	78	mylibrary/
drwxr-xr-x	90	bazel-out/
drwxr-xr-x	715	external/

Copy bb_clientd path to clipboard

Download as tarball



```
$ ls -l ~/bb_clientd/cas/mycluster/blobs/\
    directory/50260edb9db6035abf9bf368e45520d\
    b424dfa2b8612abfdcfc0eba7287faca8-245
total 0
dr-xr-xr-x 1 root root 0 Jan  1 2000 mylibrary
dr-xr-xr-x 1 root root 0 Jan  1 2000 bazel-out
dr-xr-xr-x 1 root root 0 Jan  1 2000 external
```



Q&A